

Practical 2

Min-Max Algorithm (Divide and Conquer) — Design and Analysis of Algorithms

Program (C)

```
// Aim: implement Min-Max algorithm using divide and conquer
#include <stdio.h>

struct Pair { int min, max; };

struct Pair findMinMax(int arr[], int low, int high) {
    struct Pair result, left, right;
    int mid;

    if (low == high) {
        result.min = result.max = arr[low];
        return result;
    }

    if (high == low + 1) {
        if (arr[low] < arr[high]) { result.min = arr[low]; result.max = arr[high]; }
        else { result.min = arr[high]; result.max = arr[low]; }
        return result;
    }

    mid = (low + high) / 2;
    left = findMinMax(arr, low, mid);
    right = findMinMax(arr, mid + 1, high);

    result.min = (left.min < right.min) ? left.min : right.min;
    result.max = (left.max > right.max) ? left.max : right.max;
    return result;
}

int main() {
    int arr[] = {12, 45, 3, 27, 19};
    int n = sizeof(arr) / sizeof(arr[0]);

    printf("Array elements: ");
    for (int i = 0; i < n; i++) printf("%d ", arr[i]);
    printf("\n");

    struct Pair result = findMinMax(arr, 0, n - 1);

    printf("Minimum element = %d\n", result.min);
    printf("Maximum element = %d\n", result.max);

    return 0;
}
```

Output

```
Array elements: 12 45 3 27 19
Minimum element = 3
Maximum element = 45
```